

Can a Single NVIDIA L20 Sustain a 70B-Class LLM?

An Empirical Study of Qwen2.5-72B AWQ Marlin Serving

Kevin Li

Kevin-Li-2025/llm-quant-bench

Draft: May 20, 2026

Abstract

Serving 70B-class open-weight language models is commonly associated with 80GB accelerators or multi-GPU inference, yet many practical deployments are constrained to smaller 48GB-class GPUs. This report evaluates whether a single NVIDIA L20 can sustain a 70B-class quantized model under a fixed production-style serving workload. We serve Qwen2.5-72B-Instruct-AWQ using vLLM 0.8.5.post1 with AWQ Marlin on one L20 GPU and measure throughput, latency, reliability, and GPU-board energy over both short sweeps and a 24-hour soak. Under a fixed workload of approximately 512 input tokens, 256 output tokens, and concurrency 10, the system completed 36,740 out of 36,740 requests over 86,412.6 seconds, sustaining 108.84 output tokens/s with p95 time-to-first-token (TTFT) of 6.61s, p95 end-to-end latency of 23.54s, no request failures, and no CUDA OOM/error signatures in the vLLM log. GPU power sampled every 10 seconds yielded an estimated 330.15W average board power, 7.92 kWh GPU-board energy, 0.330 output tokens/J, and 1.008 total tokens/J.

The result should be interpreted narrowly: it is strong evidence that a single L20 can sustain one carefully configured Qwen2.5-72B AWQ serving workload, not evidence of lossless quality retention, long-context high-concurrency behavior, or general production SLA coverage. Relative to the closest public 512-input/256-output/10-concurrent Qwen 2.5 72B Q4 table we found, this L20 run is competitive with and above the published single-GPU figures, while NVIDIA’s optimized NIM profiles for Qwen2.5 72B on L20 are multi-GPU FP8 configurations. We conclude that single-L20 70B-class AWQ serving is feasible and measurable, and that the next necessary step is quality-retention evaluation against a high-quality FP16/BF16 or trusted external baseline.

1 Introduction

Large open-weight language models in the 70B-parameter class occupy a useful point in the model landscape: they are substantially stronger than many 7B–14B systems, can be self-hosted, and are small enough to be realistic for organizations that cannot deploy frontier-scale clusters. The operational difficulty is that these models are usually discussed in the context of 80GB GPUs or multi-GPU servers. A 48GB GPU has enough memory for 4-bit weights, but whether it can sustain useful serving behavior over time depends on more than static model fit. KV-cache allocation, batching policy, quantization kernels, prompt/output shape, request concurrency, and thermal or memory-fragmentation effects all determine whether the result is an engineering demo or a repeatable service.

This report studies a concrete question:

Can one NVIDIA L20 sustain a 70B-class quantized LLM under a fixed, externally comparable serving workload for 24 hours?

We answer this question empirically for Qwen2.5-72B-Instruct-AWQ, served through vLLM with the AWQ Marlin kernel. We focus on sustained serving behavior rather than model quality. The model family itself is already well documented: Qwen2.5-72B-Instruct is a 72.7B-parameter instruction-tuned model with grouped-query attention and long-context support, and the Qwen team publishes both model cards and speed benchmarks for several runtimes and quantization modes [7, 8, 9]. The contribution of this report is not a new model or quantization method. The contribution is a transparent, reproducible measurement of a constrained single-GPU deployment point that is often discussed informally but rarely documented with day-long stability and energy numbers.

Contributions.

- We report a 24-hour fixed-shape serving soak of Qwen2.5-72B-Instruct-AWQ on a single NVIDIA L20, including success rate, TTFT, end-to-end latency, output throughput, request throughput, GPU-board energy, and OOM/error checks.
- We provide the exact serving configuration, workload shape, sampling controls, energy formula, and measurement scope.
- We compare the result against public Qwen 2.5 72B/Q4 serving numbers, Qwen official speed benchmarks, and NVIDIA NIM hardware profile guidance, while explicitly separating comparable aggregate-throughput data from non-comparable single-request speed tables.
- We identify the limits of the claim and the experiments required before making quality-retention or production-SLA claims.

2 Background

2.1 Why 70B on 48GB Is Nontrivial

Weight-only 4-bit quantization can reduce a 70B-class model’s weight footprint enough to fit inside a 48GB GPU. Static fit, however, does not imply sustained serving capacity. During autoregressive decoding, memory is also consumed by KV cache, runtime buffers, scheduler state, CUDA graphs or eager-mode buffers, and fragmentation. The available KV-cache budget depends on the selected context length, maximum sequence count, and batching policy. A service that fits one request at a short context length may fail at longer context, and a service that survives a short benchmark can still degrade under a day-long stream of requests.

This distinction matters for user-facing claims. “Runs on a single L20” can mean a one-request smoke test, a low-throughput interactive endpoint, a high-throughput batched endpoint, or a stable day-long service. These are different claims. This report targets the third and fourth meanings under a fixed 512/256 serving shape.

2.2 vLLM, Continuous Batching, and PagedAttention

vLLM is a high-throughput serving engine built around efficient KV-cache management and continuous batching. Its PagedAttention paper reports 2–4× throughput improvements over prior serving systems under comparable latency conditions [3]. NVIDIA GenAI-Perf and vLLM serving benchmarks both emphasize the same core serving metrics: output token throughput, request throughput, TTFT, inter-token latency, and end-to-end request latency [5, 10]. We follow this metric vocabulary.

Continuous batching is central to interpreting our result. The per-request decode speed in the 24-hour run is only about 11–15 tokens/s, but the aggregate service produces 108.84 output tokens/s because multiple requests are decoded concurrently. Therefore, this result should not be described as a single-user decode speed. It is aggregate server throughput at concurrency 10.

2.3 AWQ and Marlin

Activation-aware Weight Quantization (AWQ) is a post-training weight-only quantization approach that protects salient channels identified through activation statistics, enabling 4-bit deployment while preserving much of the model behavior [4]. Marlin is a mixed-precision FP16×INT4 inference kernel designed to recover close to the ideal speedup of 4-bit weight compression over useful batch sizes; the Marlin paper reports that integration with vLLM can improve end-to-end inference speed [1]. Our evaluated stack uses vLLM’s AWQ Marlin path (`awq_marlin`), which is important because a generic 4-bit model file and an optimized 4-bit serving kernel are not equivalent deployment configurations.

3 Experimental Setup

3.1 Hardware and Software

The test machine exposed one NVIDIA L20 GPU with approximately 46.1GB visible VRAM through ‘nvidia-smi’. The model weights were stored locally on the remote host. The serving environment used:

- Model: Qwen/Qwen2.5-72B-Instruct-AWQ.
- Runtime: vLLM 0.8.5.post1.
- Quantization path: AWQ Marlin (`--quantization awq_marlin`).
- Serving API: OpenAI-compatible `/v1/chat/completions`.
- PyTorch/CUDA stack: PyTorch 2.6.0 + CUDA 12.4 runtime in the vLLM environment.

The primary service configuration was:

```
vllm serve /home/hhai/models/Qwen2.5-72B-Instruct-AWQ \
  --host 0.0.0.0 --port 8001 \
  --served-model-name qwen72b-awq-l20 \
  --quantization awq_marlin \
  --dtype half \
  --max-model-len 1024 \
  --gpu-memory-utilization 0.98 \
  --max-num-seqs 48 \
  --max-num-batched-tokens 4096 \
  --enforce-eager \
  --swap-space 1 \
  --disable-log-requests \
  --trust-remote-code
```

The short-context configuration intentionally caps `max_model_len` at 1024. This is not a long-context service profile; it is a high-throughput fixed-shape profile. Separate 4096- and 8192-context single-concurrency validation runs are reported in Section 4.4.

3.2 Workload

The externally comparable fixed-shape benchmark used:

- Prompt set: 128 unique prompts.
- Raw tokenizer prompt length: 498 tokens.
- Server-side prompt usage after chat formatting: approximately 527 tokens on average.
- Output control: `max_tokens=256, min_tokens=256, ignore_eos=true`.
- Sampling: `temperature=0`.
- Streaming: enabled, with `stream_options.include_usage=true` to collect prompt and completion token counts from server-side usage metadata.

The 24-hour soak reused this fixed-shape workload at concurrency 10. This shape was selected because it aligns closely with the public GigaGPU 2026 benchmark methodology, which reports 512-token prompts, 256-token generations, and 10 concurrent users for Qwen 2.5 72B Q4 on several GPUs [2].

3.3 Metrics

We use standard serving metrics:

$$\text{success_rate} = \frac{N_{\text{successful}}}{N_{\text{total}}}, \quad (1)$$

$$\text{output_tok/s} = \frac{\sum_i \text{output_tokens}_i}{T_{\text{benchmark}}}, \quad (2)$$

$$\text{req/s} = \frac{N_{\text{completed}}}{T_{\text{benchmark}}}. \quad (3)$$

TTFT is the time from request submission to the first streamed token. End-to-end latency is the time from request submission to final response. Inter-token latency (ITL) and time per output token (TPOT) describe the decode phase after the first token. These definitions are consistent with NVIDIA GenAI-Perf’s metric descriptions and vLLM’s serving benchmark output [5, 10].

Energy is computed from ‘nvidia-smi’ GPU board-power samples:

$$E_J = \int P(t) dt, \quad (4)$$

$$\text{output_tok/J} = \frac{\sum_i \text{output_tokens}_i}{E_J}, \quad (5)$$

$$\text{total_tok/J} = \frac{\sum_i (\text{prompt_tokens}_i + \text{output_tokens}_i)}{E_J}. \quad (6)$$

The implementation uses trapezoidal integration over the sampled power trace. Power was sampled approximately every 10 seconds. These are GPU-board energy estimates, not full-system wall-power measurements; CPU, DRAM, storage, power-supply losses, and cooling overhead are not included.

4 Results

4.1 Fixed-Shape Concurrency Sweep

Table 1 shows the short fixed-shape sweep. Throughput increases from concurrency 1 to concurrency 16, while tail latency rises. Concurrency 24 is a saturation point: throughput falls below the concurrency-16 result and p95 latency increases sharply. This is consistent with the expected throughput-latency tradeoff in batched LLM serving.

Concurrency	Requests	Success	p95 TTFT	p95 Lat.	Out tok/s	Req/s	p05 decode
1	8	100%	0.76s	15.88s	16.21	0.063	16.92
4	32	100%	2.94s	18.63s	57.10	0.223	14.99
8	64	100%	6.06s	22.20s	93.38	0.365	12.13
10	80	100%	6.71s	23.69s	108.70	0.425	11.26
16	128	100%	11.01s	36.29s	127.70	0.499	7.66
24	120	100%	37.26s	64.08s	120.51	0.471	5.25

Table 1: Fixed-shape sweep for approximately 512 input tokens and 256 output tokens. The concurrency-16 run produced the highest short-run throughput, while concurrency 24 saturated and regressed.

The operational interpretation is not that concurrency 16 is always preferable. It is preferable only if the service can tolerate p95 end-to-end latency around 36s for this workload. For a more balanced throughput-latency point, concurrency 10 is more defensible and maps to the public 10-concurrent benchmark shape used for comparison.

4.2 24-Hour Soak

Table 2 reports the 24-hour run. The benchmark duration was 86,412.60 seconds. The load process completed normally and wrote `load_summary.json`. Both `stdout/stderr` logs were inspected, and the vLLM service log was searched for CUDA OOM, OutOfMemory, Traceback, ERROR, and Killed signatures; none were found for the run.

Two observations are important. First, the 24-hour throughput matches the short concurrency-10 screening run: 108.84 output tok/s over 24 hours versus 108.70 output tok/s in the short run. This suggests no obvious thermal, memory-fragmentation, or scheduler degradation under the tested workload. Second, the p95 TTFT and p95 latency are stable but not low. This configuration is best understood as a throughput-oriented service profile, not a low-latency chat profile.

4.3 Energy

Table 3 summarizes GPU-board energy. The average power, 330.15W, is higher than some public nominal L20 TDP listings, but it is the observed board-power value reported by `nvidia-smi` under our workload. We report it directly rather than normalizing to a spec-sheet power value.

Metric	Value
Duration	86,412.60s
Requests	36,740
Successful requests	36,740
Failed requests	0
Success rate	100%
Prompt tokens	19,361,980
Output tokens	9,405,440
Total tokens	28,767,420
Output throughput	108.84 output tok/s
Request throughput	0.425 req/s
p50 TTFT	6.55s
p95 TTFT	6.61s
p99 TTFT	6.62s
p50 latency	23.52s
p95 latency	23.54s
p99 latency	23.55s
p05 per-request decode speed	11.26 tok/s
p50 per-request decode speed	15.08 tok/s
p95 per-request decode speed	15.14 tok/s
CUDA OOM/error signatures	0

Table 2: Twenty-four-hour fixed-shape soak at concurrency 10. The short concurrency-10 screening run measured 108.70 output tok/s; the 24-hour run sustained 108.84 output tok/s, indicating no measurable throughput decay under this workload.

The energy result is useful for regression testing and deployment planning, but should not be used as a complete data-center energy estimate. Full-system wall power can be materially higher depending on CPU, memory, fans, power supply, and cooling.

4.4 Long-Context Capacity Checks

We also ran single-concurrency capacity checks at longer contexts. These are not the same workload as the fixed-shape throughput run, but they make the single-L20 70B claim more concrete.

The service can handle 4K and 8K-class prompts at concurrency 1 without OOM in these tests, but the measured throughput is much lower than the 1024-context fixed-shape serving profile. A credible high-concurrency 8K or 16K claim would require a separate service configuration and a separate soak.

5 External Positioning

Table 5 compares our result to the closest public numbers found during the literature and web search. These comparisons are directional. Serving throughput is sensitive to model variant, quantization format, kernel, scheduler, prompt length, output length, concurrency, sampling settings, and measurement method.

The most comparable public table is GigaGPU’s April 2026 benchmark, which states that it used vLLM 0.8.x with continuous batching, 512-token prompts, 256-token generations, and 10 concurrent users [2]. Under that shape, their Qwen 2.5 72B Q4 entries report 32 tok/s on RTX 3090, 58–82 tok/s on RTX 5090 variants, and 45 tok/s on RTX 6000 Pro. Our L20 result is above

Metric	Value
Power samples	8,618
Power trace duration	86,410.0s
Average GPU board power	330.15W
GPU board energy	28,528,468J
GPU board energy	7.92 kWh
Output tokens/J	0.330
Total tokens/J	1.008
Joules/output token	3.033
Joules/total token	0.992
Output tokens/kWh	1,186,870
Total tokens/kWh	3,630,153

Table 3: GPU-board energy estimate from 10-second ‘nvidia-smi’ power samples. This is not wall-power energy.

Context	Prompt tokens	Concurrency	Success	p95 TTFT	Out tok/s	OOM
4096	3,875	1	5/5	5.28s	10.02	No
8192	7,514	1	3/3	11.03s	6.16	No

Table 4: Single-concurrency long-context validation runs. These are capacity checks, not high-throughput long-context service results.

that range and adds a day-long stability measurement. This does not prove that L20 is generally faster than those GPUs; it shows that this particular L20/AWQ-Marlin/vLLM configuration is highly competitive under the tested shape.

The Qwen official speed benchmark is valuable for sanity checking, but it is not the same test. Qwen reports single-request 72B AWQ vLLM speed on two A100 80GB GPUs at batch size 1 and long 2048-token outputs, with speeds from 44.30 tok/s at input length 1 to 30.02 tok/s at input length 30,720 [8]. Those numbers measure a different operating point: high-quality multi-GPU single-request speed and long-context behavior, not aggregate 10-concurrent serving on a 48GB card.

NVIDIA NIM’s supported-model table is also instructive. Its optimized Qwen2.5 72B Instruct L20 FP8 profiles list 4 or 8 L20 GPUs, while A100 SXM BF16 profiles list 4 or 8 GPUs depending on profile [6]. Our result therefore demonstrates a different deployment path: single-L20 AWQ Marlin outside the conservative optimized FP8 profile matrix.

6 Discussion

6.1 Why the Result Is Strong

The strongest aspect of the result is not the absolute throughput alone. It is the agreement between the short concurrency-10 screening run and the day-long run. A short run at 108.70 output tok/s could be a transient peak. A 24-hour run at 108.84 output tok/s with zero failures and no OOM/error signatures is evidence of sustained stability under that exact workload.

The result is also practically meaningful because it uses a single 48GB-class GPU. Many public 70B serving configurations rely on 80GB GPUs or multi-GPU tensor parallelism. A single-GPU deployment avoids inter-GPU communication, simplifies scheduling, and reduces operational com-

Source	Hardware	Model / Quant	Shape	Result
This work	1x L20 48GB	Qwen2.5-72B AWQ Marlin	512/256, c10, 24h	108.84
GigaGPU Apr. 2026	1x RTX 3090	Qwen 2.5 72B Q4	512/256, c10	32
GigaGPU Apr. 2026	1x RTX 5090	Qwen 2.5 72B Q4	512/256, c10	58-82
GigaGPU Apr. 2026	1x RTX 6000 Pro	Qwen 2.5 72B Q4	512/256, c10	45
Qwen official	1x A100 80GB	Qwen2.5-72B AWQ, Transformers	input 1, output 2048, batch 1	11.50
Qwen official	1x A100 80GB	Qwen2.5-72B AWQ, Transformers	input 6144, output 2048, batch 1	8.17
Qwen official	1x A100 80GB	Qwen2.5-72B AWQ, Transformers	input 14336, output 2048, batch 1	5.57
Qwen official	2x A100 80GB	Qwen2.5-72B AWQ, vLLM	input 1, output 2048, batch 1	44.30
Qwen official	2x A100 80GB	Qwen2.5-72B AWQ, vLLM	input 6144, output 2048, batch 1	40.67
Qwen official	2x A100 80GB	Qwen2.5-72B AWQ, vLLM	input 14336, output 2048, batch 1	36.63
Qwen official	2x A100 80GB	Qwen2.5-72B AWQ, vLLM	input 30720, output 2048, batch 1	30.02

Table 5: Directional external comparison. Results are output tokens/s or reported tokens/s from the source. The GigaGPU rows use a similar 512/256/concurrency-10 aggregate-throughput shape. The Qwen official rows are single-request speed references and should not be directly compared to aggregate concurrency-10 throughput.

plexity. The tradeoff is that model length, concurrency, and latency must be tightly controlled.

6.2 Throughput Versus Latency

The service is throughput-oriented. At concurrency 10, p95 TTFT is 6.61s and p95 latency is 23.54s for 256-token outputs. This may be acceptable for batch generation, background summarization, asynchronous agents, or internal processing pipelines. It is less appropriate for low-latency interactive chat where users expect the first token in under a second. Reducing concurrency, shortening inputs or outputs, enabling a different scheduler profile, or using a smaller model would improve latency at the cost of aggregate throughput.

6.3 The Role of Context Length

The main result uses `max_model_len` set to 1024. This is a deliberate throughput configuration. Qwen2.5-72B supports much longer contexts, but long context shifts the bottleneck by increasing prefill cost and KV-cache pressure. The 4K and 8K single-concurrency checks show feasibility, not throughput. A separate paper-quality long-context claim should use fixed 4K/8K/16K workloads, concurrency sweeps, and independent soak tests.

6.4 What This Does Not Prove

The result does not prove that the AWQ model is lossless relative to FP16/BF16. It does not prove that all 70B models fit or perform similarly on L20. It does not prove production reliability across arbitrary prompts, tool use, long outputs, multi-tenant traffic, or long-context retrieval workloads. It also does not prove full-system energy efficiency, because we measured GPU board power rather than wall power.

These limitations are not cosmetic; they define the next experiments. A strong systems result becomes a stronger research result when paired with quality retention and broader deployment ablations.

7 Threats to Validity

Single model and quantization path. All main results use Qwen2.5-72B-Instruct-AWQ with AWQ Marlin. GPTQ, GGUF, FP8, EXL2, and other 70B-class models may behave differently.

Fixed workload. The 24-hour workload uses approximately 512 input tokens and exactly 256 output tokens with concurrency 10. Real traffic has variable prompt lengths, output lengths, retries, user disconnects, tool calls, and burstiness.

Quality not evaluated. The report measures serving capacity and stability. It does not evaluate MMLU, CMMLU, GSM8K, MT-Bench, LongBench, or domain-specific quality retention against an FP16/BF16 baseline.

Energy scope. Energy is GPU-board energy from `nvidia-smi`, not wall energy. Board-power sampling is useful for regression but incomplete for cost or emissions accounting.

No multi-run confidence intervals. The 24-hour run is a single day-long run. Repeating the run across days, hosts, driver versions, and ambient conditions would improve statistical confidence.

8 Future Work

The next stage should prioritize quality retention:

- Run MMLU/MMLU-Pro, CMMLU/C-Eval, GSM8K/MATH, MT-Bench/Arena-Hard, and LongBench/LongBench v2.
- Compare the AWQ candidate against a high-quality full-precision Qwen2.5-72B endpoint or a trusted hosted reference. A single L20 cannot host that baseline locally.
- Add additional 70B-class models: Llama 3.3 70B AWQ, DeepSeek-R1-Distill-Llama-70B AWQ, and Qwen3 70B-class checkpoints if suitable quantized variants are available.
- Compare runtimes: vLLM, SGLang, llama.cpp/GGUF, and TensorRT-LLM where compatible.
- Add quantization ablations: AWQ Marlin, GPTQ Marlin, FP8 where hardware and profiles permit, and GGUF Q4 variants.
- Measure wall power in addition to GPU-board power, and report cost per million output tokens under realistic electricity pricing.

9 Conclusion

This report shows that a single NVIDIA L20 can sustain Qwen2.5-72B-Instruct-AWQ serving for 24 hours under a fixed 512-input/256-output, concurrency-10 workload when configured with vLLM 0.8.5.post1 and AWQ Marlin. The system completed 36,740 requests with 100% success, sustained 108.84 output tokens/s, consumed an estimated 7.92 kWh of GPU-board energy, and showed no OOM/error signatures. The result is strong evidence for a specific single-L20 70B-class throughput-serving configuration. It should be framed as sustained serving evidence, not as a lossless-quality or general-production claim. The most important next step is to pair this systems result with rigorous quality-retention evaluation.

A Reproducibility Checklist

- Repository: <https://github.com/Kevin-Li-2025/llm-quant-bench>
- Result commit: 20558b4 Document 24h L20 soak results.
- Remote root: /home/hhai/llm-quant-bench.
- Run path: runs/qwen72b-awq-l20/ soak-24h-fixed512x256-c10-20260519T034856Z.
- Load summary: load/load_summary.json.
- Power log: gpu_power.csv.
- Energy summary: energy_summary.json.

Benchmark command:

```
python3 -m llm_quant_bench load \
  --config runs/qwen72b-awq-l20/config.fixed512x256.json \
  --dataset runs/qwen72b-awq-l20/fixed-512in-256out.jsonl \
  --out runs/qwen72b-awq-l20/soak-24h-fixed512x256-c10-.../load \
  --concurrency 10 \
  --duration-seconds 86400 \
  --stream-usage
```

Energy command:

```
python3 scripts/summarize_energy.py \
  --summary .../load/load_summary.json \
  --power-log .../gpu_power.csv \
  --out .../energy_summary.json
```

B Recommended arXiv Readiness Checklist

Before submission, the report should add:

- A quality-retention section with baseline-vs-AWQ results.

- A complete hardware table including driver version, CUDA version, CPU, RAM, disk, and host power measurement if available.
- At least one repeated shorter run to establish run-to-run variance.
- A public artifact bundle or redacted sample of the fixed-shape prompt generator.
- A stronger related-work section covering LLM serving, quantization, and energy-aware inference.

References

- [1] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. Marlin: Mixed-precision auto-regressive parallel inference on large language models. *arXiv preprint arXiv:2408.11743*, 2024.
- [2] GigaGPU. Tokens/sec benchmark update, april 2026. <https://gigagpu.com/tokens-sec-benchmark-update-april-2026/>, 2026. Accessed 2026-05-20.
- [3] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*, 2023.
- [4] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Xingyu Chen, and Song Han. Awq: Activation-aware weight quantization for llm compression and acceleration. *Proceedings of Machine Learning and Systems*, 2024.
- [5] NVIDIA. Nvidia genai-perf: Metrics and benchmarking documentation. https://docs.nvidia.com/deeplearning/triton-inference-server/archives/triton-inference-server-2500/user-guide/docs/perf_analyzer/genai-perf/README.html, 2025. Accessed 2026-05-20.
- [6] NVIDIA. Nvidia nim for large language models: Supported models. <https://docs.nvidia.com/nim/large-language-models/1.14.0/supported-models.html>, 2026. Accessed 2026-05-20.
- [7] Qwen Team. Qwen2.5-72b-instruct model card. <https://huggingface.co/Qwen/Qwen2.5-72B-Instruct>, 2024. Accessed 2026-05-20.
- [8] Qwen Team. Qwen2.5 speed benchmark. https://qwen.readthedocs.io/en/v2.5/benchmark/speed_benchmark.html, 2024. Accessed 2026-05-20.
- [9] Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.
- [10] vLLM Project. vllm serving benchmark documentation. <https://docs.vllm.ai/en/latest/cli/bench/serve.html>, 2026. Accessed 2026-05-20.